KARNATAKA STATE OPEN UNIVERSITY

MUKTHAGANGOTRI, MYSORE- 570 006

DEPARTMENT OF STUDIES IN INFORMATION TECHNOLOGY

M.Sc IN INFORMATION TECHNOLOGY

III SEMESTER

ELECTIVE 1

DISTRIBUTED SYSTEMS

MSIT- 116D

MSIT-116D:
Distributed Systems

Course Design and Editorial Committee

Prof. M.G.Krishnan
Vice Chancellor
Karnataka State Open University
Mukthagangotri, Mysore – 570 006

Prof. Vikram Raj Urs
Dean (Academic) & Convener
Karnataka State Open University
Mukthagangotri, Mysore – 570 006

Head of the Department and Course Co-Ordinator

Rashmi B.S
Assistant Professor & Chairperson
DoS in Information Technology
Karnataka State Open University
Mukthagangotri, Mysore – 570 006

Course Editor

Ms. Nandini H.M
Assistant professor of Information Technology
DoS in Information Technology
Karnataka State Open University
Mukthagangotri, Mysore – 570 006

Course Writers

Dr. Suresha
Professor
DoS in Computer Science,
University of Mysore, Mysore.

Smt. L. Hamsaveni
Assistant Professor
DoS in Computer Science,
University of Mysore, Mysore.

Publisher

Registrar
Karnataka State Open University
Mukthagangotri, Mysore – 570 006

Developed by Academic Section, KSOU, Mysore
Karnataka State Open University, 2014

Further information on the Karnataka State Open University Programmes may be obtained from the University's Office at Mukthagangotri, Mysore – 6.

Printed and Published on behalf of Karnataka State Open University, Mysore-6 by the **Registrar (Administration)**

Karnataka State Open University

Mukthagangothri, Mysore – 570 006

Third Semester M.Sc in Information Technology

MSIT 116D : Distributed Systems

Preface

The word *distributed* in terms such as "distributed system", "distributed programming", and "distributed algorithm" originally referred to computer networks where individual computers were physically distributed within some geographical area. The terms are nowadays used in a much wider sense, even referring to autonomous processes that run on the same physical computer and interact with each other by message passing. While there is no single definition of a distributed system, the following defining properties are commonly used:

- There are several autonomous computational entities, each of which has its own local memory.
- The entities communicate with each other by message passing.

In this article, the computational entities are called *computers* or *nodes*.

A distributed system may have a common goal, such as solving a large computational problem. Alternatively, each computer may have its own user with individual needs, and the purpose of the distributed system is to coordinate the use of shared resources or provide communication services to the users.

Organization of the material: The book introduces its topics in ascending order of complexity and is divided into four modules, containing four units each.

In the first module, we begin with an introduction to Distributed Systems, System Models and Architecture of distributed systems.

In the second module, we discussed Inter process communication, External Data Representation and Remote Procedure Call concepts.

The third module contains concepts of Operating System and overview of Security techniques with Digital Signatures.

In the fourth module, File Service Architecture, Name Services, Peer to Peer Systems are discussed.

Happy reading to all the students!!!

UNIT 1: Introduction to Distributed Systems

Structure:

1.0 OBJECTIVES

At the end of this unit you will be able to know:

- Definition of distributed systems
- History of distributed systems
- Benefits of resource sharing
- Challenges of distributed systems.

1.1 INTRODUCTION

This unit is about the introduction to distributed systems. We start with definition of distributed systems. It lists a list of examples of distributed systems. It also introduces Resource Sharing and Challenges of distributed systems. The goal is to provide motivational examples of contemporary distributed systems, illustrating both the

pervasive role of distributed systems and the great diversity of the associated applications.

1.2 DEFINITION

A distributed system can be defined as a "collection of independent computers that appear to the users of the system as a single computer". The computers in a distributed system are essentially independent machines. This means that, architecturally, the machines are capable of operating independently. The software enables this set of connected machines to appear as a single computer to the users of the system.

Another definition: A distributed system can be defined as a system of networked computers that coordinate their activity only by message passing.

In a distributed system, each computer has its own memory, has its own clock, and each computer runs its own operating systems.

Why distributed systems?

It is mainly because of availability of powerful yet cheap microprocessors (PCs, workstations, PDAs, embedded systems, etc.) and continuing advances in communication technology.

The benefits of distributed systems:

Price/performance ratio: You don't get twice the performance for twice the price in buying computers. Processors are only so fast and the price/performance curve becomes nonlinear and steep very quickly. With multiple CPUs, we can get (almost) double the performance for double the money (as long as we can figure out how to keep the processors busy and the overhead negligible).

Distributing machines: It makes sense to put the CPUs for ATM cash machines at the source, each networked with the bank. Each bank can have one or more computers networked with each other and with other banks. For computer graphics, it makes sense to put the graphics processing at the user's terminal to maximize the bandwidth between the device and processor.

Computer supported cooperative networking: Users that are geographically separated can now work and play together. Examples of this are electronic whiteboards, distributed document systems, audio/video teleconferencing, email, file transfer, and games such as Doom, Quake, Age of Empires, and Duke Nuke'em, Starcraft, and scores of others.

Increased reliability: If a small percentage of machines break, the rest of the system remains intact and can do useful work.

Incremental growth: A company may buy a computer. Eventually the workload is too great for the machine. The only option is to replace the computer with a faster one. Networking allows you to add on to an existing infrastructure.

Remote services: Users may need to access information held by others at their systems. Examples of this include web browsing, remote file access, and programs such as Napster and Gnutella to access MP3 music.

Mobility: Users move around with their laptop computers, Palm Pilots, and WAP phones. It is not feasible for them to carry all the information they need with them.

Advantages and disadvantages:

A distributed system has distinct advantages over a set of non-networked smaller computers. Data can be shared dynamically – giving private copies (via floppy disk, for example) does not work if the data is changing. Peripherals can also be shared. Some peripherals are expensive and/or infrequently used so it is not justifiable to give each PC

a peripheral. These peripherals include optical and tape jukeboxes, typesetters, large format color printers and expensive drum scanners. Machines themselves can be shared and workload can be distributed amongst idle machines. The advantages are Economics, Speed, Inherent distribution, Reliability, Incremental growth.

There are many problems with distributed systems: Designing, implementing and using distributed software may be difficult. Issues of creating operating systems and/or languages that support distributed systems arise. The network may lose messages and/or become overloaded. Rewiring the network can be costly and difficult. Security becomes a far greater concern. Easy and convenient data access from anywhere creates security problems. The disadvantages are software, network, more components to fail, security

1.3. EXAMPLES OF DISTRIBUTED SYSTEMS

There many examples of distributed systems. They are: the Internet and the associated World Wide Web, web search, online gaming, email, social networks, eCommerce, etc. Distributed systems encompass many of the most significant technological developments of recent years and hence an understanding of the underlying technology is absolutely central to knowledge of modern computing.

We now look at more specific examples of distributed systems to further illustrate the diversity and indeed complexity of distributed systems provision today.

Web search: Web search has emerged as a major growth industry in the last decade, with recent figures indicating that the global number of searches has risen to over 10 billion per calendar month. The task of a web search engine is to index the entire contents of the World Wide Web, encompassing a wide range of information styles including web pages, multimedia sources and (scanned) books. This is a very complex task, as current estimates state that the Web consists of over 63 billion pages and one trillion unique web addresses. Given that most search engines analyze the entire web content and then carry

out sophisticated processing on this enormous database, this task itself represents a major challenge for distributed systems design.

Mobile and ubiquitous computing: Technological advances in device miniaturization and wireless networking have led increasingly to the integration of small and portable computing devices into distributed systems. These devices include:

- Laptop computers.
- Handheld devices, including mobile phones, smart phones, GPS-enabled devices, pagers, personal digital assistants (PDAs), video cameras and digital cameras.
- Wearable devices, such as smart watches with functionality similar to a PDA.
- Devices embedded in appliances such as washing machines, hi-fi systems, cars and refrigerators.

The portability of many of these devices, together with their ability to connect conveniently to networks in different places, makes *mobile computing* possible. Mobile computing is the performance of computing tasks while the user is on the move, or visiting places other than their usual environment. In mobile computing, users who are away from their 'home' intranet (the intranet at work, or their residence) are still provided with access to resources via the devices they carry with them.

Wide area network applications: There are many wide area network applications, namely email (electronic mail), bbs (bulletin board systems), netnews (group discussions on single subject), gopher (text retrieval service), WWW (world wide web, is the biggest example of distributed system).

The Internet: network of networks, global access to "everybody" (data, service, other actor; open ended), enormous size (open ended), no single authority

Intranet: it is a portion of the internet managed by an organization. A single authority , protected access.

Key characteristics of distributed systems

The following are the key characteristics of distributed systems.

- Resource sharing
- Openness
- Concurrency
- Scalability
- Fault Tolerance
- Transparency
- No global clock
- Independent failures

1.4. RESOURCE SHARING

Note that the users are so accustomed to the benefits of resource sharing that they may easily overlook their significance. We routinely share hardware resources such as printers, data resources such as files, and resources with more specific functionality such as search engines.

When we look at from the point of view of hardware provision, we share equipment such as printers and disks to reduce costs. But of far greater significance to users is the sharing of the higher-level resources that play a part in their applications and in their everyday work and social activities. For example, users are concerned with sharing data in the form of a shared database or a set of web pages. Similarly, users think in terms of shared resources such as a search engine or a currency converter, without regard for the server or servers that provide these.

We use the term *service* for a distinct part of a computer system that manages a collection of related resources and presents their functionality to users and applications.
The only access we have to the service is via the set of operations that it exports.

The fact that services restrict resource access to a well-defined set of operations is in part standard software engineering practice. For effective sharing, each resource must be managed by a program that offers a communication interface enabling the resource to be accessed and updated reliably and consistently.
The term *server* refers to a running program (a *process*) on a networked computer that accepts requests from programs running on other computers to perform a service and responds appropriately. The requesting processes are referred to as *clients*, and the overall approach is known as *client-server computing*. In this approach, requests are sent in messages from clients to a server and replies are sent in messages from the server to the clients. A complete interaction between a client and a server, from the point when the client sends its request to when it receives the server's response, is called a *remote invocation*.

Many, but certainly not all, distributed systems can be constructed entirely in the form of interacting clients and servers. The World Wide Web, email and networked printers all fit this model.

There are two types of resources: they are hardware resources (e.g., disks and printers) and software resources (e.g., files, windows, and data objects).

Hardware sharing is used for convenience and reduction of cost. Where as Data sharing (shared usage of information) is used for consistency (compilers and libraries), exchange of information (database), and cooperative work (groupware).

Service resources are used as search engines, computer-supported cooperative working and so on.

Resource Manager: Software module that manages a set of resources. Each resource requires its own management policies and methods.

Client server model: server processes act as resource managers for a set of resources and

a set of clients.

Object based model: resources are objects that can move. Object manager is movable. Request for a task on an object is sent to the current manager. Manager must be co-located with object.

Examples for distributed systems in use: ARGUS, Amoeba, Mach, Arjuna, Clouds, Emerald

Quality of service

Once users are provided with the functionality that they require of a service, such as the file service in a distributed system, we can go on to ask about the quality of the service provided. The main nonfunctional properties of systems that affect the quality of the service experienced by clients and users are *reliability*, *security* and *performance*. *Adaptability* to meet changing system configurations and resource availability has been recognized as a further important aspect of service quality.

1.5. CHALLENGES OF DISTRIBUTED SYSTEMS

In this section we describe the main challenges of distributed systems.

Heterogeneity

The Internet enables users to access services and run applications over a heterogeneous collection of computers and networks. Heterogeneity (that is, variety and difference) applies to all of the following:

- Networks;
- Computer hardware;
- Operating systems;
- Programming languages;
- Implementations by different developers.

Although the Internet consists of many different sorts of network , their differences are masked by the fact that all of the computers attached to them use the Internet protocols to communicate with one another.

Openness

The openness of a computer system is the characteristic that determines whether the system can be extended and re-implemented in various ways. The openness of distributed systems is determined primarily by the degree to which new resource-sharing services can be added and be made available for use by a variety of client programs.

Openness cannot be achieved unless the specification and documentation of the key software interfaces of the components of a system are made available to software developers.

A further benefit that is often cited for open systems is their independence from individual vendors.

To summarize:

- Open systems are characterized by the fact that their key interfaces are published.
- Open distributed systems are based on the provision of a uniform communication mechanism and published interfaces for access to shared resources.
- Open distributed systems can be constructed from heterogeneous hardware and software, possibly from different vendors.

Security

Many of the information resources that are made available and maintained in distributed systems have a high intrinsic value to their users. Their security is therefore of considerable importance. Security for information resources has three components: confidentiality (protection against disclosure to unauthorized individuals), integrity (protection against alteration or corruption), and availability (protection against interference with the means to access the resources).

In a distributed system, clients send requests to access data managed by servers, which involves sending information in messages over a network. For example:
1. A doctor might request access to hospital patient data or send additions to that data.
2. In electronic commerce and banking, users send their credit card numbers across the Internet.

In both examples, the challenge is to send sensitive information in a message over a network in a secure manner.
However, security challenges have not yet been fully met: for *'Denial of service attacks'* and *'Security of mobile code'.*

Scalability
Distributed systems operate effectively and efficiently at many different scales, ranging from a small intranet to the Internet. A system is described as *scalable* if it will remain effective when there is a significant increase in the number of resources and the number of users. The number of computers and servers in the Internet has increased dramatically.

The design of scalable distributed systems presents the following challenges:

- *Controlling the cost of physical resources*
- *Controlling the performance loss*
- *Preventing software resources running out*
- *Avoiding performance bottlenecks*

Some shared resources are accessed very frequently; for example, many users may access the same web page, causing a decline in performance. Ideally, the system and application software should not need to change when the scale of the system increases, but this is difficult to achieve. The issue of scale is a dominant theme in the development of distributed systems.

Failure handling

Computer systems sometimes fail. When faults occur in hardware or software, programs may produce incorrect results or may stop before they have completed the intended computation.

Failures in a distributed system are partial – that is, some components fail while others continue to function. Therefore the handling of failures is particularly difficult.
The following techniques for dealing with failures are discussed throughout the book:

Detecting failures: Some failures can be detected. For example, checksums can be used to detect corrupted data in a message or a file.

.

Masking failures: Some failures that have been detected can be hidden or made less severe. Two examples of hiding failures:

1. Messages can be retransmitted when they fail to arrive.
2. File data can be written to a pair of disks so that if one is corrupted, the other may still be correct.

Tolerating failures: Most of the services in the Internet do exhibit failures – it would not be practical for them to attempt to detect and hide all of the failures that might occur in such a large network with so many components. Their clients can be designed to tolerate failures, which generally involve the users tolerating them as well.

Recovery from failures: Recovery involves the design of software so that the state of permanent data can be recovered or 'rolled back' after a server has crashed. In general, the computations performed by some programs will be incomplete when a fault occurs, and the permanent data that they update (files and other material stored in permanent storage) may not be in a consistent state.

Redundancy: Services can be made to tolerate failures by the use of redundant components. Consider the following examples:

1. There should always be at least two different routes between any two routers in the Internet.
2. In the Domain Name System, every name table is replicated in at least two different servers.
3. A database may be replicated in several servers to ensure that the data remains accessible after the failure of any single server; the servers can be designed to detect faults in their peers; when a fault is detected in one server, clients are redirected to the remaining servers.

Distributed systems provide a high degree of availability in the face of hardware faults. The *availability* of a system is a measure of the proportion of time that it is available for use. When one of the components in a distributed system fails, only the work that was using the failed component is affected. A user may move to another computer if the one that they were using fails; a server process can be started on another computer.

Concurrency

Both services and applications provide resources that can be shared by clients in a distributed system. There is therefore a possibility that several clients will attempt to access a shared resource at the same time. For example, a data structure that records bids for an auction may be accessed very frequently when it gets close to the deadline time. In this case, their operations on the shared resource may conflict with one another and produce inconsistent results.

A shared resource in a distributed system must be responsible for ensuring that it operates correctly in a concurrent environment. This applies not only to servers but also to objects in applications. Therefore any programmer who takes an implementation of an object that was not intended for use in a distributed system must do whatever is necessary to make it safe in a concurrent environment. For an object to be safe in a concurrent environment, its operations must be synchronized in such a way that its data remains consistent. This

can be achieved by standard techniques such as semaphores, which are used in most operating systems.

Transparency

Transparency is defined as the concealment from the user and the application programmer of the separation of components in a distributed system, so that the system is perceived as a whole rather than as a collection of independent components.

The various transparencies are:

Access transparency: enables local and remote resources to be accessed using identical operations.
Location transparency: enables resources to be accessed without knowledge of their physical or network location (for example, which building or IP address).
Concurrency transparency: enables several processes to operate concurrently using shared resources without interference between them.
Replication transparency: enables multiple instances of resources to be used to increase reliability and performance without knowledge of the replicas by users or application programmers.
Failure transparency: enables the concealment of faults, allowing users and application programs to complete their tasks despite the failure of hardware or software components.
Mobility transparency: allows the movement of resources and clients within a system without affecting the operation of users or programs.
Performance transparency: allows the system to be reconfigured to improve performance as loads vary.
Scaling transparency: allows the system and applications to expand in scale without change to the system structure or the application algorithms.

1.6. SUMMARY

In this unit we introduced distributed systems. We started with definition of distributed systems. We presented a list of examples of distributed systems. This unit also introduced Resource Sharing and Challenges of distributed systems. In this unit we also discussed about different transparencies.

1.7. KEYWORDS

A distributed system: can be defined as a "collection of independent computers that appear to the users of the system as a single computer".

Resource Manager: Software module that manages a set of resources. Each resource requires its own management policies and methods.

Transparency: is defined as the concealment from the user and the application programmer of the separation of components in a distributed system, so that the system is perceived as a whole rather than as a collection of independent components

1.8. UNIT-END EXERCISES AND ANSWERS

1. Give definition of distributed systems.
2. Give some examples of distributed systems.
3. Explain the characteristics of distributed systems.
4. Discuss about resource sharing.
5. What are the challenges of distributed systems? Explain
6. What is transparency? Explain various transparency.

Answers: SEE

1. 1.2
2. 1.3
3. 1.3

4. 1.4
5. 1.5
6. 1.5

1.9 SUGGESTED READINGS

- Distributed Systems: Concepts and Design By: George Coulouris, Jean Dollimore, Tim Kindberg.
- DISTRIBUTED SYSTEMS Principles and Paradigm By : Andrew S. Tanenbaum , Maarten Van Steen

UNIT 2: System Models

Structure:

2. 0 OBJECTIVES

At the end of this unit you will be able to know about:

- Design of distributed systems
- Various system models

2. 1 INTRODUCTION

This unit provides an explanation of three important and complementary ways in which the design of distributed systems can usefully be described and discussed. The purpose of the System models is to illustrate/describe common properties and design choices for distributed system in a single descriptive model.

There are three types of models: namely,

1. Physical models consider the types of computers and devices that constitute a system and their interconnectivity, without details of specific technologies.

Physical models: capture the hardware composition of a system in terms of computers and other devices and their interconnecting network.

2. Architectural models describe a system in terms of the computational and communication tasks performed by its computational elements; the computational elements being individual computers or aggregates of them supported by appropriate network interconnections. Client-server and peer-to-peer are two of the most commonly used forms of architectural model for distributed systems.
3. Fundamental models take an abstract perspective in order to describe solutions to individual issues faced by most distributed systems.

There is no global time in a distributed system, so the clocks on different computers do not necessarily give the same time as one another. All communication between processes is achieved by means of messages. Message communication over a computer network can be affected by delays, can suffer from a variety of failures and is vulnerable to security attacks. These issues are addressed by three models: namely, the interaction model, the failure model, and the security model.

2. 2 PHYSICAL MODELS

A physical model is a representation of the underlying hardware elements of a distributed system that abstracts away from specific details of the computer and networking technologies employed. The end result is a physical architecture with a significant increase in the level of heterogeneity embracing, for example, the tiniest embedded devices utilized in ubiquitous computing through to complex computational elements found in Grid computing. These systems deploy an increasingly varied set of networking technologies and offer a wide variety of applications and services. Such systems potentially involve up to hundreds of thousands of nodes.

2. 3 ARCHITECTURAL MODELS

The architecture of a system is its structure in terms of separately specified components and their interrelationships. The overall goal is to ensure that the structure will meet present and likely future demands on it. Major concerns are to make the system reliable, manageable, adaptable and cost-effective. Architecture models define the main components of the system, what their roles are and how they interact (software architecture), and how they are deployed in an underlying network of computers (system architecture). Architecture model is concerned with the placement of its parts, namely how components are mapped to underlying network and the relationship between them, that is, their functional roles and patterns of communication between them. An architectural model simplifies and abstracts the functions of the individual components of a distributed system and then it considers the placement of the components across a network of an architectural model, the interrelationships between the components.

Architectural elements:

To understand the fundamental building blocks of a distributed system, it is necessary to consider four key questions:
What are the entities that are communicating in the distributed system?
How do they communicate, or, more specifically, what communication paradigms used?
What (potentially changing) roles and responsibilities do they have in the overall architecture?
How are they mapped on to the physical distributed infrastructure (what is their placement)?

Now we examine two architectural styles stemming from the role of individual processes: client-server and peer-to-peer.

Client-server: This is the architecture that is most often cited when distributed systems are discussed. It is historically the most important and remains the most widely employed. The Figure 1.2.1 illustrates the simple structure in which processes take on the roles of being clients or servers. In particular, client processes interact with individual server processes in potentially separate host computers in order to access the shared resources that they manage.

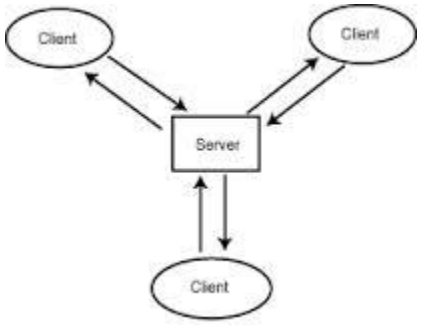


Figure 1.2.1: Client Server Model

Peer-to-peer: In this architecture all of the processes involved in a task or activity play similar roles, interacting cooperatively as peers without any distinction between client and server processes or the computers on which they run. In practical terms, all participating processes run the same program and offer the same set of interfaces to each other. While the client-server model offers a direct and relatively simple approach to the sharing of data and other resources, it scales poorly. The centralization of service provision and management implied by placing a service at a single address does not scale well beyond the capacity of the computer that hosts the service and the bandwidth of its network connections. The Figure 1.2.2 shows Peer-to-Peer model.

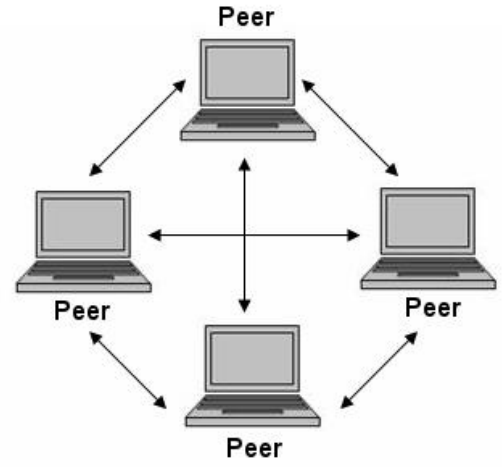


Figure 1.2.2: Peer to Peer Model

2. 4 FUNDAMENTAL **MODELS**

Fundamental models deal with formal description of the properties that is common to architecture models. Since no global time in a distributed system, so the clocks on different computers do not necessarily give the same time as one another. Messages communications can be affected by delays and suffer from a variety of failures and vulnerable to security attacks. All system models have some common fundamental properties.

There are three fundamental models: 1) interaction models, 2) failure models and 3) security models.

1 Interaction: Processes communicate with messages and coordinate via synchronization and ordering of activities. The message delays are often of considerable duration, the coordination between processes is limited by lack of global clock. The interaction model deals with performance and with difficulty of setting time limits.

2 Failure: The correct operation is threatened whenever a fault occurs in any of the computers and network. We should define types of faults in order to tolerate them for the system to continue to run correctly. The **failure** model attempts to give a precise specification of the faults that can be exhibited by processes and communication channels.

3 Security: The modular feature of distributed system and their openness exposes them to attack by both external and internal agents. Security model defines and classifies the forms of attack may take, providing a basis for the analysis of threats to a system and for the design of system that are able to resist them. The **security** model discusses the possible threats to processes and communication channels. It introduces the concept of a secure channel, which is secure against those threats

2.5 SUMMARY

In this unit, we have discussed about three types of system models namely, Physical models, Architecture models and Fundamental models.

2.6 KEYWORDS

Physical model: considers the types of computers and devices that constitute a system and their interconnectivity, without details of specific technologies

Architectural model: describes a system in terms of the computational and communication tasks performed by its computational elements.

Fundamental model: takes an abstract perspective in order to describe solutions to individual issues faced by most distributed systems

Client server model: The client–server model of computing is a distributed application structure that partitions tasks or workloads between the providers of a resource or service, called servers, and service requesters, called clients.

Peer-to-peer model: In this architecture all of the processes involved in a task or activity play similar roles, interacting cooperatively as peers without any distinction between client and server processes or the computers on which they run

2.7 UNIT-END EXERCISES AND ANSWERS

1. What are the four key questions to understand the fundamental building blocks of a distributed system?
2. Explain client server model with a neat diagram.
3. Explain peer-to-peer model with a neat diagram.
4. Compare client server model and peer-to-peer model.
5. Explain fundamental models.

Answers: SEE

1. 2.3
2. 2.3
3. 2.3
4. 2.3
5. 2.4

2.8 SUGGESTED READINGS

- Distributed Systems: Concepts and Design By: George Coulouris, Jean Dollimore, Tim Kindberg.
- DISTRIBUTED SYSTEMS Principles and Paradigm By : Andrew S. Tanenbaum , Maarten Van Steen

UNIT 3 ARCHITECTURES OF DISTRIBUTED SYSTEMS

Structure:

3.0 OBJECTIVES

At the end of this unit you will be able to know:

- Various architectures of distributed systems.
- Layered (software) Architecture for Client-Server Systems
- Overlay Networks

3.1 INTRODUCTION

This unit is introduces the various architectures of distributed systems.

In this unit we discuss the architectures suitable for distributed system**s,** We also discuss about Layered (software) Architecture for Client-Server Systems**,** System Architecture and Overlay Networks

Broadly we have the following architectures.

Software Architectures: describe the organization and interaction of software components; focuses on logical organization of software (component interaction, etc.).

System Architectures: describe the placement of software components on physical machines

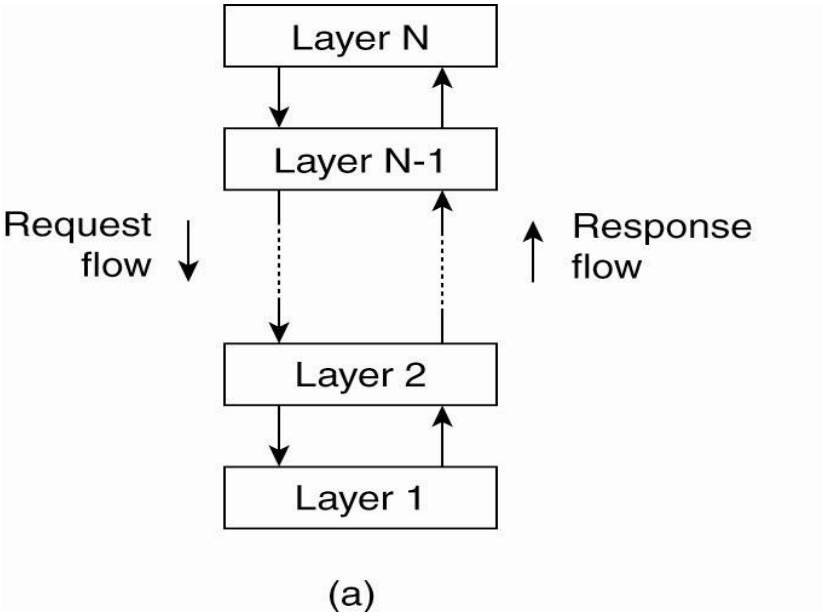
The realization of an architecture may be centralized (most components located on a single machine), decentralized (most machines have approximately the same functionality), or hybrid (some combination).

Architectural Styles: An architectural style describes a particular way to configure a collection of components and connectors. A **Component** is a module with well-defined interfaces; reusable, replaceable. A **Connector** is a communication link between modules.

3.2 THE ARCHITECTURES SUITABLE FOR DISTRIBUTED SYSTEMS

The following are the architectures suitable for distributed systems.

- Layered architectures
- Object-based architectures
- Data-centered architectures
- Event-based architectures



(a)

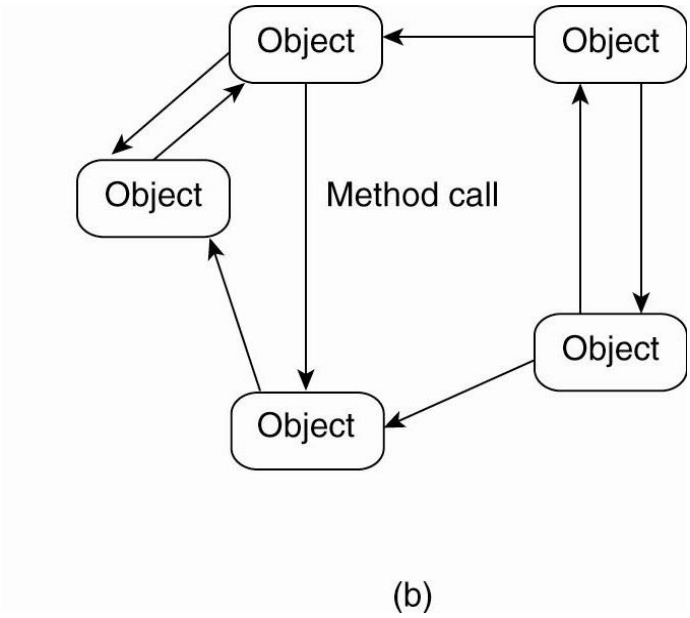


Figure 1.3.1: The (a) layered architectural style and (b) The object-based architectural style.

Data-Centered Architectures: Main purpose is data access and update. Processes interact by reading and modifying data in some shared repository in either active or passive manner.

- Traditional data base (passive): responds to requests
- Blackboard system (active): clients solve problems collaboratively; system updates clients when information changes.

Communication via event propagation, in distributed. systems seen often in Publish/ Subscribe. *e.g.*, register interest in market information; get email updates. Decouples sender and receiver, use asynchronous communication

Event-based architecture: It supports several communication styles:

- Publish-subscribe
- Broadcast
- Point-to-point

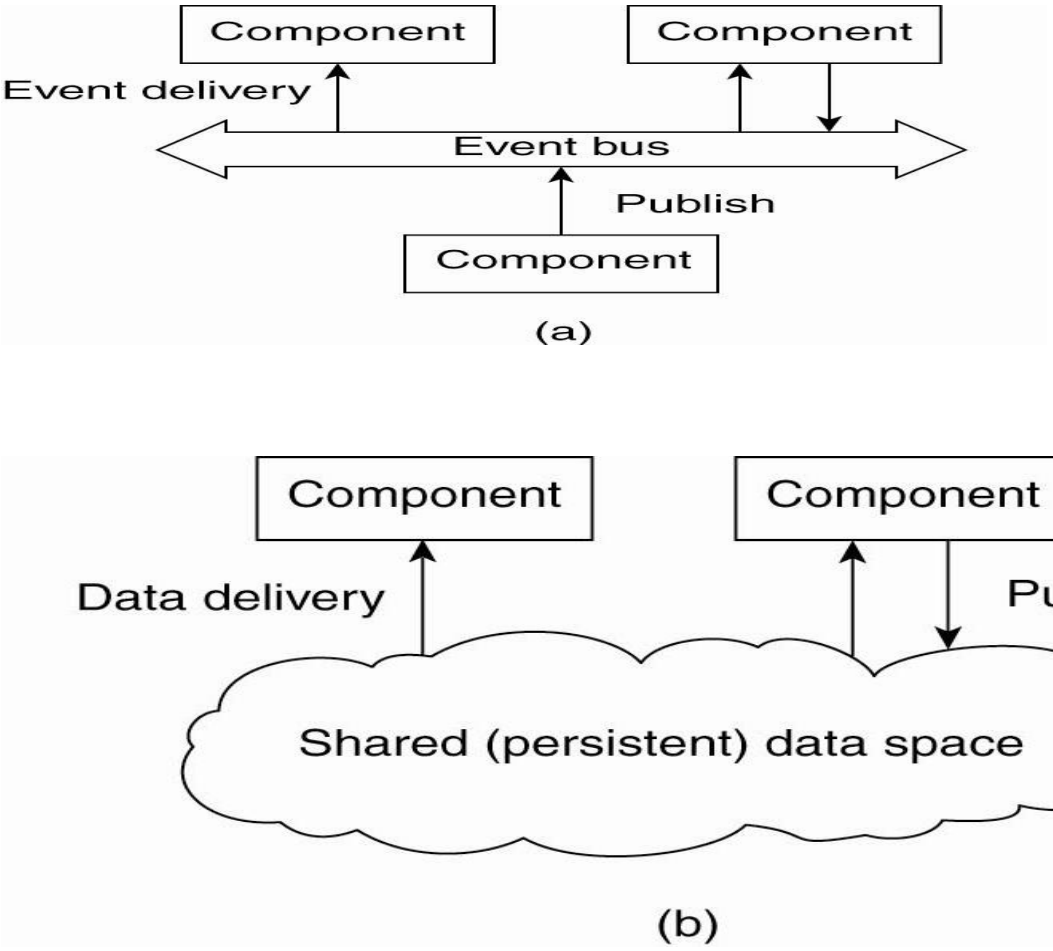


Figure 1.3.2: (a) The event-based architectural style (b) The shared data-space architectural style.

Data Centric Architecture; *e.g.*, shared distributed file systems or Web-based distributed systems

Combination of data-centered and event based architectures. Processes communicate asynchronously.

System Architectures for Distributed Systems

The system architecture for distributed systems can be either centralized or decentralized or hybrid.

Centralized: characterized by traditional client-server structure.

- Vertical (or hierarchical) organization of communication and control paths (as in layered software architectures)
- Logical separation of functions into client (requesting process) and server (responder)

Decentralized: characterized by peer-to-peer.

- Horizontal rather than hierarchical communication and control
- Communication paths are less structured; symmetric functionality.

Hybrid: It is combination of elements of Client Server and Peer-to-Peer.

Classification of a system as centralized or decentralized refers to communication and control organization, primarily.

Traditional Client-Server system: In traditional Client-Server system, the Processes are divided into two groups (clients and servers). They use Synchronous communication: request-reply protocol. In LANs often implemented with a connectionless protocol (unreliable). In WANs, communication is typically connection-oriented TCP/IP (reliable), with high likelihood of communication failures.

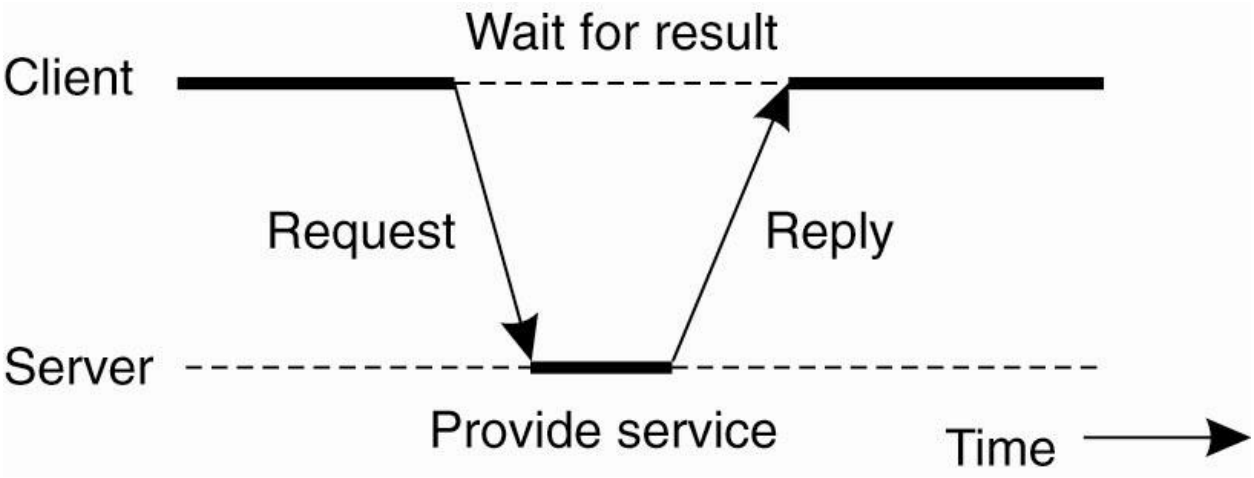


Figure 1.3.3: General interaction between a client and a server.

Transmission Failures: with connectionless transmissions, failure of any sort means no reply

The possibilities are:

- Request message was lost
- Reply message was lost
- Server failed either before, during or after performing the service

3.3 LAYERED (SOFTWARE) ARCHITECTURE FOR CLIENT-SERVER SYSTEMS

The layered architecture consists of three levels:

1) **User-interface level**: It usually uses GUIs for interacting with end users.
2) **Processing level**: It is concerned with data processing applications, which is the core functionality.
3) **Data level**: interacts with data base or file system. Data usually is persistent; exists even if no client is accessing it. It is concerned with File or database system.

The Figure 1.3.4 shows the application layering.

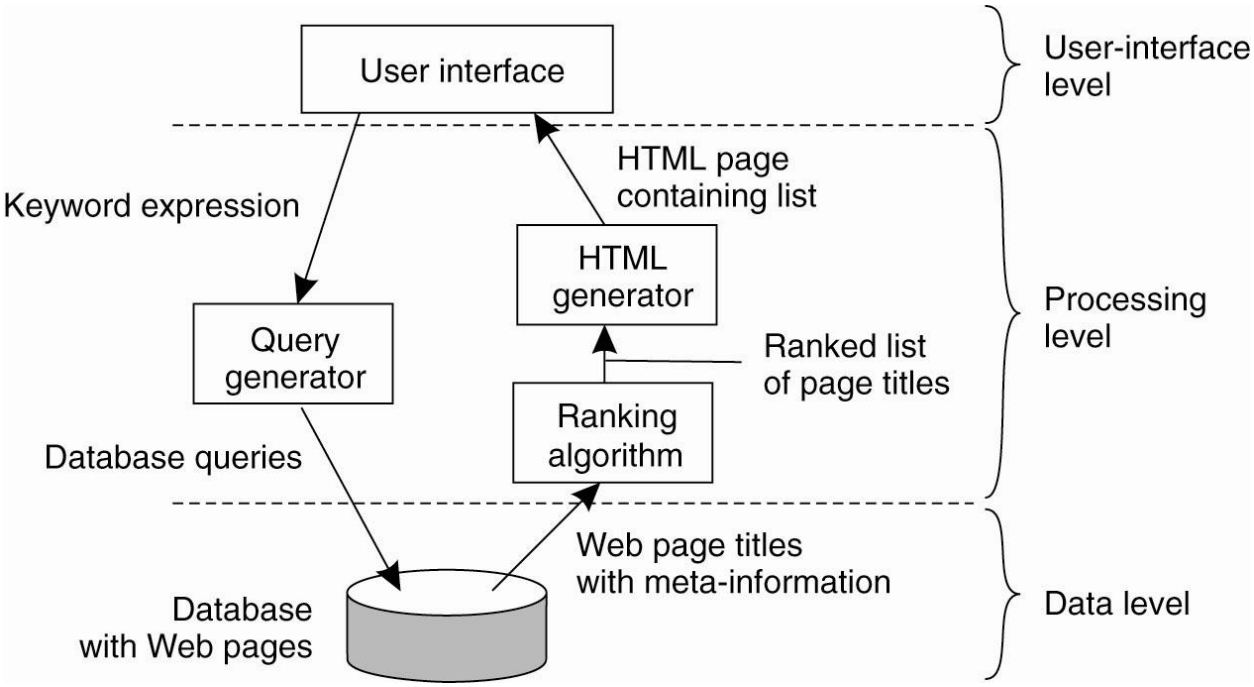


Figure 1.3.4: The simplified organization of an Internet search engine into three different layers

3.4 SYSTEM ARCHITECTURE

The system architecture involves the Mapping the software architecture to system hardware, which has close correspondence between logical software modules and actual computers

Multi-tiered architectures

Multi-tier architecture (often referred to as *n*-tier architecture) is a client–server architecture in which presentation, application processing, and data management functions are physically separated. The most widespread use of multi-tier architecture is the three-tier architecture. N-tier application architecture provides a model by which developers can create flexible and reusable applications. By segregating an application into tiers, developers acquire the option of modifying or adding a specific layer, instead of reworking the entire application. *Layer* and *tier* are roughly equivalent terms, but *layer* typically implies software and *tier* is more likely to refer to hardware. Two-tier and three-tier are the most common architectures.

Two-tiered Client-Server Architectures

Two-tiered architecture is a technological hardware and software configuration in which the presentation and application components of the architecture are resident on one system in a two-system configuration and the database component is resident on the other system in the configuration.

Thin Client approach: Server provides processing and data management; client provides simple graphical display. This increases work load at server. It is easier to manage, more reliable, client machines don't need to be so large and powerful.

Fat-Client approach: All application processing and some data reside at the client. This reduces work load at server and more scalable. But, it is harder to manage by system admin and less secure.

Three-tiered Client-Server Architectures

In some applications servers may also need to be clients, which lead to a three level architecture. Three-tiered architecture is a technological hardware and software configuration in which the presentation, application and database components of the architecture are resident on separate and distinct systems within the configuration. Some examples are Distributed transaction processing and Web servers that interact with database servers. In Three-tiered Architectures, functionality distributed across three levels of machines instead of two.

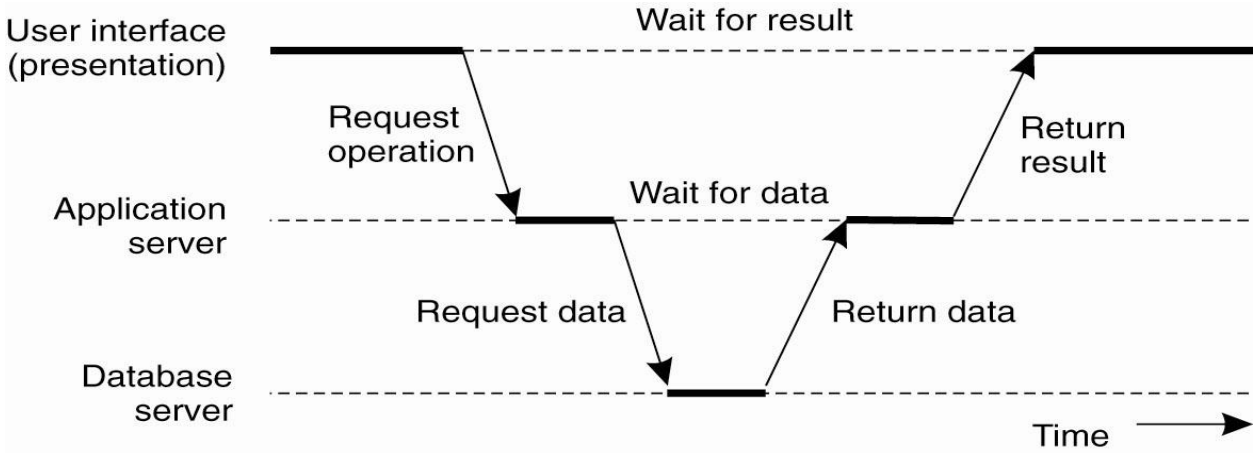


Figure 1.3.5: An example of a server acting as client.

Centralized verses Decentralized Architectures

Vertical distribution: Traditional client-server architectures exhibit **vertical distribution**. Each level serves a different purpose in the system. In this l*ogically* different components reside on different nodes.

Horizontal distribution (P2P): each node has roughly the same processing capabilities and stores/manages part of the total system data. It has better load balancing, more resistant to denial-of-service attacks, harder to manage than client-server. Communication and control is not hierarchical; all about equal.

Peer-to-Peer

A **peer-to-peer (P2P) network** is a type of decentralized and distributed network architecture in which individual nodes in the network (called *"peers"*) act as both suppliers and consumers of resources, It is a network of personal computers, each of which acts as both client and sever, so that each can exchange files and folders directly with every other computer on the network.

In Peer-to-peer Nodes act as both client and server and interaction is symmetric. Each node acts as a server for part of the total system data. Overlay networks connect nodes in the P2P system. Nodes in the overlay use their own addressing system for storing and retrieving data in the system. Nodes can route requests to locations that may not be known by the requester.

3.5 OVERLAY NETWORKS

Overlay networks are logical or *virtual* networks, built on top of a physical network. A link between two nodes in the overlay may consist of several physical links. Messages in the overlay are sent to logical addresses, not physical (IP) addresses. Various approaches used to resolve logical addresses to physical.

Each node in a P2P system knows how to contact several other nodes. The overlay network may be structured (nodes and content are connected according to some design

that simplifies later lookups) or unstructured (content is assigned to nodes without regard to the network topology).

Structured P2P Architectures

In structured P2P architectures, a common approach is to use a distributed hash table (DHT) to organize the nodes. Traditional hash functions convert a key to a hash value, which can be used as an index into a hash table. The keys are unique, each represents an object to store in the table. The hash function value is used to insert an object in the hash table and to retrieve it.

In a DHT, data objects and nodes are each assigned a key which hashes to a random number from a very large identifier space (to ensure uniqueness). A mapping function assigns objects to nodes, based on the hash function value. A lookup, also based on hash function value, returns the network address of the node that stores the requested object.

Characteristics of DHT

It is scalable to thousands, even millions of network nodes. Search time increases more slowly than size; usually $O(\log(N))$. It is also fault tolerant, able to re-organize itself when nodes fail. It is decentralized, no central coordinator.

Unstructured P2P

Unstructured P2P organizes the overlay network as a random graph. Each node knows about a subset of nodes, its "neighbors". Neighbors are chosen in different ways: physically close nodes, nodes that joined at about the same time, etc. Data items are randomly mapped to some node in the system and lookup is random.

Locating a Data Object by Flooding

Send a request to all known neighbors If not found, neighbors forward the request to their neighbors. Works well in small to medium sized networks, doesn't scale well. "Time-to-live" counter can be used to control number of hops. Example system: Gnutella & Freenet (Freenet uses a caching system to improve performance)

Structured P2P Architectures

The hybrid architectures combine client-server and P2P architectures.

For example,

- Edge-server systems; e.g. ISPs, which act as servers to their clients, but cooperate with other edge servers to host shared content
- Collaborative distributed systems; e.g., BitTorrent, which supports parallel downloading and uploading of chunks of a file. First, interact with C/S system, then operates in decentralized manner.

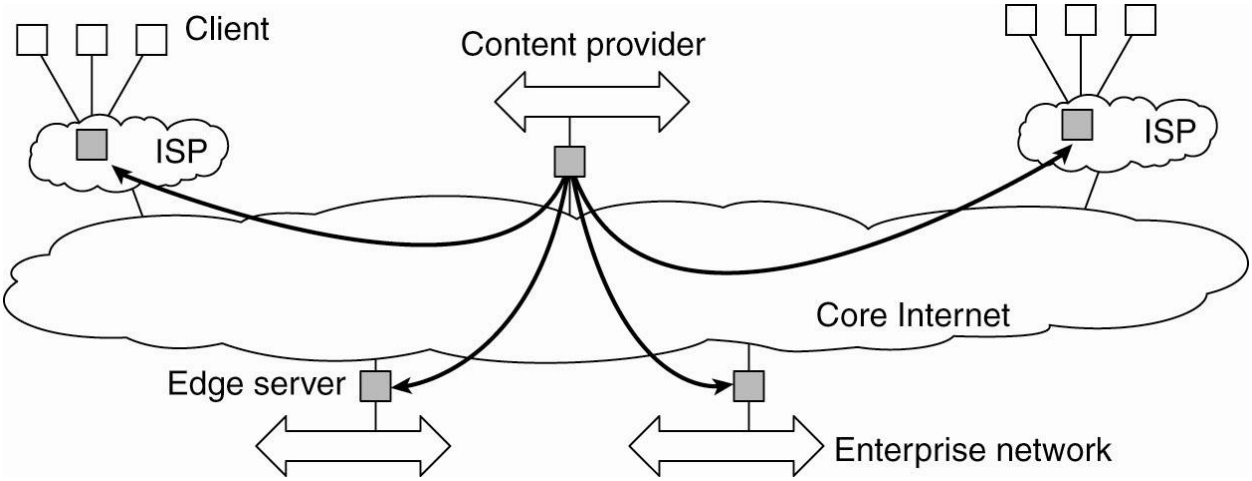


Figure 1.3.6: Viewing the Internet as consisting of a collection of edge servers.

P2P verses Client/Server

- P2P computing allows end users to communicate without a dedicated server.
- Communication is still usually synchronous (blocking)
- There is less likelihood of performance bottlenecks since communication is more distributed. Data distribution leads to workload distribution.
- Resource discovery is more difficult than in centralized client-server computing and look-up/retrieval is slower
- P2P can be more fault tolerant, more resistant to denial of service attacks because network content is distributed. Individual hosts may be unreliable, but overall, the system should maintain a consistent level of service.

3.6 SUMMARY

In this unit we introduced the various architectures of distributed systems.

We also discussed about the architectures suitable for distributed system**s,** Layered (software) Architecture for Client-Server Systems**,** System Architecture and Overlay Networks.

3.7 KEYWORDS

Hybrid: It is combination of elements of Client Server and Peer-to-Peer.

Data-Centered Architectures: Main purpose is data access and update. Processes interact by reading and modifying data in some shared repository in either active or passive manner.

Multi-tier architecture (often referred to as *n*-tier architecture) is a client–server architecture in which presentation, application processing, and data management functions are physically separated.

3.8 UNIT-END EXERCISES AND ANSWERS

1. Explain Layered (software) Architecture for Client-Server Systems.
2. Define Two-tiered C/S Architectures and Three-tiered C/S Architectures
3. Compare Structured and Unstructured P2P Architectures
4. Explain Structured P2P Architectures
5. Compare P2P verses Client/Server

Answers: SEE

1. 3.3
2. 3.4
3. 3.5
4. 3.5
5. 3.4

3.9 SUGGESTED READINGS

- Distributed Systems: Concepts and Design By: George Coulouris, Jean Dollimore, Tim Kindberg.
- DISTRIBUTED SYSTEMS Principles and Paradigm By : Andrew S. Tanenbaum , Maarten Van Steen